Computing Counterexamples to Serre's Modularity Conjecture

Rhys Aron, Catherine Fontaine, Zack Horton
with Professor Patrick Allen

July 2023

Abstract

Jean-Pierre Serre famously disproved his own conjecture outlined in his paper "Sur les représentations modulaires de degré 2 de $\mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q})$". This paper outlines an algorithm for computing counterexamples, and the results it finds.

Contents

1 Introduction **3**

2 Preliminary notions on modular forms **4**
2.1 Basic definitions 4
2.2 Hecke operators and Dirichlet character 5
2.3 Subspaces of $S_k(\Gamma_1(N))$ 7

3 Preliminary notions on representations **9**
3.1 Representations 9
3.2 Prime ideals in a ring of integers 10
3.3 Galois representations 11
3.4 Serre's conjecture 11

4 Program **12**
4.1 Weight one forms 12
4.2 Carayol Conditions 13
4.3 Sturm bound 13
4.4 Checking congruence 14

5 Pseudocode **15**

6 Examples **18**

Appendices **22**

A Original code **22**

B Instructions on downloading weight one form data **30**

1 Introduction

Fermat's last theorem states that there is no a,b,c $\in \mathbb{Z}$ such that $a^n + b^n = c^n$ for n strictly greater than 2. For many centuries, mathematicians had no idea how to prove or disprove this statement. It was only in 1993, more than 300 years after Fermat stated his theorem, that Andrew Wiles proved it using the Frey curve, Shimura-Taniyama conjecture and Serre's conjecture. [7] [3].

In short summary, the proof goes as follows. Firstly, the Frey curves are elliptic curves of the form $y^2 = x(x - a^n)(x + b^n)$. Such curves can be defined when there is a solution to $a^n + b^n = c^n$. Letting E ba a Frey curve, we can define a Galois representation $\rho_{E,n}$: $\mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q}) \to \mathrm{GL}_2(\mathbb{F}_n)$. By the Shimura-Taniyama conjecture, $\rho_{E,n}$ is isomorphic to a Galois representation from a modular form. Finally, by Serre's conjecture and Kenneth Ribet, it was shown that the weight and level were both 2. However, the space of modular forms of this weight and level is empty which gives a contradiction. [3]

Let's look more thoroughly at Serre's conjecture. Indeed, the French mathematician, Jean-Pierre Serre, first stated his conjectures on Galois representation in his letter to J.-F. Mestre and then in his paper 'Sur les représentations modulaires de degré 2 de $\mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q})$'. [13] [17] [18] There are two conjectures, the weak one and the strong one. The weak conjecture states that a Galois representation ρ: $\mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q}) \to \mathrm{GL}_2(\mathbb{F})$ where $\mathbb{F}$ is a finite field is isomorphic to a Galois representation coming from a modular form if $\det(\rho(\sigma)) = -1$ where $\sigma \in \mathrm{Gal}\big(\overline{\mathbb{Q}}/\mathbb{Q}\big)$ is complex conjugation. The strong conjecture gives the values of the level, the weight and the Dirichlet character of this modular form. [18] [3] However, Serre found a counterexample to the conjectured value for the Dirichlet character. The example he found is the newform in $S_2(\Gamma_1(13))$. Consider the Galois representation generated by the newform in $S_2(\Gamma_1(13))$ and the prime $\sqrt{-3}$ modulo $\sqrt{-3}$. This resulting Galois representation should be isomorphic to the one generated by a modular form in the space $S_2(\Gamma_0(13), \chi)$ where χ has order 2. However, this space is empty which raises a contradiction. [1]

The goal of this paper is to state further counterexamples to Serre's strong conjecture and explain the methods used to compute them. This paper starts by defining modular forms and other background information necessary to understand Serre's conjecture. Next we outline the algorithms written to compute counterexamples, and finally, list our findings.

2 Preliminary notions on modular forms

2.1 Basic definitions

Consider the matrix group $SL_2(\mathbb{Z})$, the group of 2×2 invertible matrices with integer coefficients and determinant 1. Define the group action of $SL_2(\mathbb{Z})$ on the part of the complex plane with strictly positive imaginary part denoted $\mathcal{H}$ as follows:

$\forall\ \tau \in \mathcal{H}$ and $\gamma := \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z})$, $\gamma(\tau) = \frac{a\tau+b}{c\tau+d}$ [16]

Definition 2.1. A function is holomorphic on Ω, an open subset of $\mathbb{C}$ if and only if it is infinitely differentiable and integrble on Ω.

Instead of defining modular forms on the whole group $SL_2(\mathbb{Z})$ which is quite restrictive, we can define them on general congruence subgroups. Congruence subgroups are subgroups of $SL_2(Z)$ generally denoted Γ. Three types of congruence subgroup are often used to describe modular forms.

$$\Gamma(\mathrm{N}) = \{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z}) \parallel \begin{pmatrix} a & b \\ c & d \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \bmod \mathrm{N} \}$$

$$\Gamma_0(\mathrm{N}) = \{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z}) \parallel \begin{pmatrix} a & b \\ c & d \end{pmatrix} \equiv \begin{pmatrix} * & * \\ 0 & * \end{pmatrix} \bmod \mathrm{N} \}$$

$$\Gamma_1(\mathrm{N}) = \{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z}) \parallel \begin{pmatrix} a & b \\ c & d \end{pmatrix} \equiv \begin{pmatrix} 1 & * \\ 0 & 1 \end{pmatrix} \bmod \mathrm{N} \}$$ [5]

Definition 2.2. A fundamental domain D for Γ, a congruence subgroup of $SL_2(\mathbb{Z})$, is the set of points in $\mathcal{H}$ such that $\forall\ \gamma \in \Gamma$, $a, b \in D$, $\gamma(a) = \gamma(b)$ if and only if $a = b$.

When viewing $\mathcal{H}$ as a subset of the complex projective plane $\mathbb{C}^*$, we can define cusps as follows:

Definition 2.3. A cusp for a congruence subgroup Γ is a point that is on the boundary of both the fundamental domain for Γ and the boundary of $\mathcal{H}$.

Note that the set of cusps is a subset of $\mathbb{Q} \cup \{\infty\}$.

Definition 2.4. A modular form of weight k and level N is a holomorphic function f on $\mathcal{H}$ such that

$f(\gamma(\tau)) = f(\ \frac{a\tau+b}{c\tau+d}\ ) = (\mathrm{c}\tau + \mathrm{d})^k\ f(\tau)$, $\forall \gamma := \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \Gamma(N)$ [16]

The holomorphic requirement ensures every modular form f has a Laurent expansion (also referred to as q-expansion) of the form $\sum_{n=0}^{\infty} a_n q^n$, where $q = e^{2\pi i}$. [16]

Definition 2.5. A cusp form for a congruence subgroup Γ is a modular form f such that $f(\tau)$ approaches 0 as τ approaches any cusp of Γ. Equivalently, cusp form is a modular form such that the $a_0 = 0$ [16]

Definition 2.6. Let $\mathrm{M}_k(\Gamma(N))$ refer to the set of weight k, level N modular forms with respect to congruence subgroup Γ. Note that the set of weight k forms with respect to $\mathrm{SL}_2(\mathbb{Z})$ is the set $\mathrm{M}_k(\Gamma(1))$. Similarly, let $\mathrm{S}_k(\Gamma(N))$ refer to the set of weight k, level N cusp forms.

Note that $\mathrm{M}_k(\Gamma(N))$ is a vector space over $\mathbb{C}$, and $\mathrm{S}_k(\Gamma(N)) \subseteq \mathrm{M}_k(\Gamma(N))$ is a subspace.

2.2 Hecke operators and Dirichlet character

Given that $\mathrm{M}_k(\Gamma(N))$ and $\mathrm{S}_k(\Gamma(N))$ have vector space structure, it is natural to define linear operators on them. Two such operators are particularly useful: the diamond and T_p operator. To understand their definition, we must first define the double coset operation of $\mathrm{GL}_2^+(\mathbb{Q})$, where $\mathrm{GL}_2^+(\mathbb{Q})$ is the group of 2×2 invertible matrices with rational coefficients and positive determinant.[5]

Definition 2.7. Let Γ_1 and Γ_2 be congruence subgroups of $SL_2(\mathbb{Z})$ and $\alpha \in \mathrm{GL}_2^+(\mathbb{Q})$. A double coset is the set $\Gamma_1\alpha\Gamma_2 = \{\gamma_1\alpha\gamma_2 \parallel \gamma_1 \in \Gamma_1 \text{ and } \gamma_2 \in \Gamma_2\}$[5]

We can define a group action of Γ_1 on $\Gamma_1\alpha\Gamma_2$. Orbits are of the form $\Gamma_1\beta$, with representatives $\beta = \gamma_1\alpha\gamma_2$ for $\gamma_1 \in \Gamma_1$ and $\gamma_2 \in \Gamma_2$. Using above notation, set $\Gamma_3 = \alpha^{-1}\Gamma_1\alpha \cap \Gamma_2$. It can be viewed as a subgroup of Γ_2 or a coset space $\Gamma_3\backslash\Gamma_2 = \{\Gamma_3\gamma_2 \parallel \gamma_2 \in \Gamma_2\}$. [5]

Remark 2.1. *The orbit space of $\Gamma_1\backslash\Gamma_1\alpha\Gamma_2$ and the coset space of $\Gamma_3\backslash\Gamma_2$ are in bijection. $\{\gamma_{2,j}\}$ are representatives for $\Gamma_3\backslash\Gamma_2$ if and only if $\{\alpha\gamma_{2,j}\}$ are representatives for $\Gamma_1\backslash\Gamma_1\alpha\Gamma_2$. [5]*

The weight k $\Gamma_1\alpha\Gamma_2$ operator takes $f \in \mathrm{M}_k(\Gamma_1)$ to $f[\Gamma_1\alpha\Gamma_2]_k = \sum_j f[\beta_j]_k$ where $f[\beta_j]_k(\tau) = (\det\beta_j)^{k-1}(\mathrm{c}_j\tau + \mathrm{d}_j)^{-k} f(\beta_j(\tau))$ for $\beta_j = \begin{pmatrix} a_j & b_j \\ c_j & d_j \end{pmatrix} \in \mathrm{GL}_2^+(\mathbb{Q})$ and the β_j are the orbit representatives for the action $\Gamma_1\backslash\Gamma_1\alpha\Gamma_2$. This operator is called the double coset operator. [5]

Remark 2.2. *$f[\alpha\beta]_k = (f[\alpha]_k)[\beta]_k$ for $\alpha, \beta \in GL_2^+(\mathbb{Q})$[5]*

Proposition 2.1. *The double coset operator does not depend on the choice of orbit representatives.*

Proof. Let $\{\beta_j\}$ and $\{\beta'_j\}$ be two sets of orbit representatives. Then, $f[\Gamma_1\alpha\Gamma_2]_k = \sum_j f[\beta_j]_k$ and $f[\Gamma_1\alpha\Gamma_2]_k = \sum_j f[\beta'_j]_k$. We know $\Gamma_1\beta_1 = \Gamma_1\beta'_1$ up to reordering. $\beta_1 = \gamma_1\alpha\gamma_2$ and $\beta'_1 = \gamma'_1\alpha\gamma'_2$, then $\alpha\gamma_2 \in \Gamma_1\alpha\gamma'_2$, so $\alpha\gamma_2 = \gamma_1"\alpha\gamma'_2$. Therefore, $f[\beta_1]_k = f[\gamma_1\alpha\gamma_2]_k = f[\alpha\gamma_2]_k = f[\gamma_1"\alpha\gamma'_2]_k = f[\alpha\gamma'_2]_k = f[\gamma'_1\alpha\gamma'_2]_k = f[\beta'_1]_k$. Indeed adding an element of Γ_1 to the weight k operator doesn't affect its value because $f \in \mathrm{M}_k(\Gamma_1)$. By induction, $\sum_j f[\beta_j]_k = \sum_j f[\beta'_j]_k$. □

We can now define the diamond and T_p operator.

Definition 2.8 (Diamond operator $\langle d\rangle$)**.** Let $f \in \mathrm{M}_k(\Gamma_1(\mathrm{N}))$ and $d \in \mathbb{Z}/\mathrm{N}\mathbb{Z}$. $\langle d\rangle f = f[\alpha]_k$ for any $\alpha = \begin{pmatrix} a & b \\ c & \delta \end{pmatrix} \in \Gamma_0(\mathrm{N})$ and $\delta \equiv \mathrm{d} \pmod{\mathrm{N}}$. If $\gcd(d, \mathrm{N})>1$, $\langle d\rangle := 0$ [5]

Proposition 2.2. *Let* $\alpha_1 = \begin{pmatrix} a_1 & b_1 \\ c_1 & d_1 \end{pmatrix}$ *and* $\alpha_2 = \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix}$, $\alpha_1, \alpha_2 \in \Gamma_0(N)$, *and* $f \in M_k(\Gamma_1(N))$. *If* $d_1 \equiv d_2 \mod N$, *then* $f[\alpha_1]_k = f[\alpha_2]_k$

Proof. Let $\alpha_1 = \begin{pmatrix} a_1 & b_1 \\ c_1 & d_1 \end{pmatrix}$ and $\alpha_2 = \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix}$ such that $\alpha_1, \alpha_2 \in \Gamma_0(\mathrm{N})$ and $d_1 \equiv d_2 \pmod{\mathrm{N}}$. We can define the map ϕ: $\Gamma_0(\mathrm{N}) \to (\mathbb{Z}/\mathrm{N}\mathbb{Z})^*$ such that $\phi(\begin{pmatrix} a & b \\ c & d \end{pmatrix}) = \mathrm{d} \pmod{\mathrm{N}}$.

We will now prove this map is surjective. Let $\mathrm{z} \in (\mathbb{Z}/\mathrm{N}\mathbb{Z})^*$. We know z has a multiplicative inverse denoted z^{-1} such that $\mathrm{z}z^{-1}=1 \bmod \mathrm{N}$. Let $\mathrm{z}z^{-1}= 1 + \mathrm{N}l$ for some $l \in \mathbb{Z}$. Consider the matrix $\alpha = \begin{pmatrix} z^{-1} & 1 \\ Nl & z \end{pmatrix}$. $\det(\alpha) = 1$, so α is invertible and the bottom left entry is clearly 0 mod N. Then, $\alpha \in \Gamma_0(\mathrm{N})$.

Let's prove that $\ker(\phi) = \Gamma_1(\mathrm{N})$.
$\subseteq$: Let $\gamma = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \ker(\phi)$. Then $\mathrm{d} \equiv 1 \bmod \mathrm{N}$, let $\mathrm{d} = \mathrm{N}k + 1$ for $k \in \mathbb{Z}$. We know $\mathrm{c} \equiv 0 \bmod \mathrm{N}$ then let $\mathrm{c} = \mathrm{N}l$ for some $l \in \mathbb{Z}$. Then, we must have that ad - bc = $\mathrm{a}(\mathrm{N}k + 1) + \mathrm{bN}l = 1$ for the determinant to be equal to 1. This forces $\mathrm{a} \equiv 1 \bmod \mathrm{N}$. Therefore, $\gamma \in \Gamma_1(\mathrm{N})$.
$\supseteq$: Holds by the definition.

Then, we have that $\Gamma_0(\mathrm{N})/\Gamma_1(\mathrm{N}) \cong (\mathbb{Z}/\mathrm{N}\mathbb{Z})^*$ and sending $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ to d (mod

N) is an isomorphism. Thus, the cosets depend on the value of the bottom right entry mod N. Then, we have that α_1 and α_2 are in the same coset and $\exists\ \gamma \in \Gamma_1(\mathrm{N})$ such that $\alpha_1 = \alpha_2\ \gamma$. Since $f \in \mathrm{M}_k(\Gamma_1(\mathrm{N}))$, $f[\alpha_1]_k{=}f[\alpha_2\gamma]_k{=}f[\alpha_2]_k$. $\square$

Definition 2.9 (T_p operator)**.** Let p be prime. Define

$$T_p : M_k\left(\Gamma_1\left(N\right)\right) \to M_k\left(\Gamma_1\left(N\right)\right)$$

$$T_p f = f\left[\Gamma_1\left(N\right)\begin{pmatrix}1 & 0\\ 0 & p\end{pmatrix}\Gamma_1\left(N\right)\right]_k$$

Working from the definition allows us to determine T_n for n composite. We state without proof that for T_{p^r} where p is prime, $r > 1$, and $n = \prod p_i^{e_i}$, we have

$$T_{p^r} = T_p T_{p^{r-1}} - p^{k-1}\langle p\rangle T_{p^{r-2}},$$

$$T_n = \prod T_{p_i^{e_i}}$$

[5] Also note that $S_k\left(\Gamma_1\left(N\right)\right)$ is an invariant subspace under $\langle d\rangle$ and T_p.

2.3 Subspaces of $S_k\left(\Gamma_1\left(N\right)\right)$

$S_k\left(\Gamma_1\left(N\right)\right)$ can be decomposed into natural subspaces with useful properties. This section discusses in broad terms how to deconstruct each space and outlines some of the most useful properties. We begin this section with a proposition.

Proposition 2.3. *For all $M \mid N$, $S_k(\Gamma_1(M)) \subseteq S_k(\Gamma_1(N))$.*

Proof. Let M divide N, let $f \in \mathrm{S}_k(\Gamma_1(\mathrm{M}))$. Since f is a modular form it is holomorphic. Also, since $\Gamma_1(\mathrm{N}) \subseteq \Gamma_1(\mathrm{M})$, $\forall\ \alpha = \begin{pmatrix}a & b\\ c & d\end{pmatrix} \in \Gamma_1(\mathrm{N})$ and $\tau \in \mathbb{C}$, $f(\alpha\tau) = (\mathrm{c}\tau + \mathrm{d})^k\ f(\tau)$.

We will now prove that f is zero at all cusp of $\Gamma_1(\mathrm{N})$. Firstly, we will prove that for any congruence subgroup H of $\mathrm{SL}_2(\mathbb{Z})$, they have a finite number of cusps. By definition, $[\mathrm{SL}_2(\mathbb{Z}){:}\mathrm{H}] = \mathrm{d} < \infty$. Let S be the set of cusps for H. $\forall\ \mathrm{s} \in \mathrm{S}$, s is also a cusp for $\mathrm{SL}_2(\mathbb{Z})$ since H is a subgroup of $\mathrm{SL}_2(\mathbb{Z})$. All the cusps of $\mathrm{SL}_2(\mathbb{Z})$ are equivalent to 0. Thus, $\forall\ \mathrm{s} \in \mathrm{S}$, $\exists$ a g *in* $\mathrm{SL}_2(\mathbb{Z})$ such that g acting on s gives 0. In addition, the partition by cosets of $\mathrm{SL}_2(\mathbb{Z})$ gives that $\mathrm{g} = \gamma_i\lambda$ for some $\lambda \in \mathrm{H}$ and γ_i is a coset representative. Since the index of H in $\mathrm{SL}_2(\mathbb{Z})$ is d, there are d such γ_i.

Therefore,
$\forall$ s $\in$ S, 0 = g(s) = $\gamma_i\lambda$(s)
$\Rightarrow$ Since γ_i is invertible, $\gamma_i^{-1}(0) = \lambda$(s)
$\Rightarrow$ s is equivalent to one of the d values $\gamma_i^{-1}(0)$ and each of the d γ_i^{-1} give a single value by the invertibility of γ_i
$\Rightarrow$ All s $\in$ S are equivalent to at most d distinct complex numbers
$\Rightarrow$ There are at most d cusps in S that are not equivalent under H
$\Rightarrow$ There is a finite number of cusps for H up to equivalence

Let s be a cusp for Γ_1(N). By the proof above, s is in some equivalence class of cusps for Γ_1(M). Let f be in $S_k(\Gamma_1(M))$. We know that f is zero at all cusps of Γ_1(M)), and thus is zero at s. Since s and f were arbitrary, we see that all functions in $S_k(\Gamma_1(M))$ are zero at all cusps of $S_k(\Gamma_1(N))$ □

This suggests a natural way to decompose $S_k(\Gamma_1(N))$ is by "old" forms, namely those also present in lower levels, and "new" forms, those not previously present. To make this notion rigorous, let $\alpha_d = \begin{pmatrix} d & 0 \\ 0 & 1 \end{pmatrix}$ with $d|N/M$. This induces a map from $S_k(\Gamma_1(M))$ to $S_k(\Gamma_1(N))$ given by $f \to f[\alpha_d]_k(\tau)$ $= d^{k-1}f(d\tau)$, allowing us to lift forms from a lower level to a higher level. Therefore, we can define the following map:

$$\iota_d : S_k(\Gamma_1(Nd^{-1}))^2 \to S_k(\Gamma_1(N))$$
$$\iota_d(f,g) = f + g[\alpha_d]_k$$

The space of oldforms, denoted $S_k(\Gamma_1(N))^{old}$, is then given by [5]

$$\sum_{p|Nprime} \iota_p(S_k(\Gamma_1(Np^{-1}))^2)$$

The space of newforms is defined as the orthogonal complement of $S_k(\Gamma_1(N))^{old}$ under an inner product on $S_k(\Gamma_1(N))$ called the Petersson inner product. The actual formula is not essential in the understanding of this paper, but we will still define it for general interest. Let $f, g \in S_k(\Gamma)$ for some congruence subgroup Γ. We first define X(Γ) as the modular curve of Γ which corresponds to the orbit space $\{\Gamma\tau : \tau \in \mathcal{H}\}$. X($\Gamma$) is compact and is a Riemann surface. We can define a volume on it equal to $V_\Gamma = \int_{X(\Gamma)} \frac{dxdy}{y^2}(\tau)$ where τ = x + yi $\in \mathcal{H}$. This inner product is denoted by $\langle f, g\rangle_\Gamma$ and is given by $\int_{X(\Gamma)} f(\tau)\overline{g(\tau)}(Im(\tau))^k \frac{dxdy}{y^2}(\tau)$ where $\overline{g(\tau)}$ corresponds to the conjugate of $g(\tau)$. Thus, the newform space is defined as the orthogonal complement under

the Petersson inner product of the oldform space denoted $\mathrm{S}_k(\Gamma_1(\mathrm{N}))^{new}$. [5] This construction matches our intuition, as it is precisely the set of forms that aren't lifted from a lower level.

We can further decompose $S_k(\Gamma_1(N))$ into eigenspaces under both the T_p and $\langle n\rangle$ operators.

Definition 2.10. A simultaneous eigenform (also referred to as simply eigenform) is an element of $S_k(\Gamma_1(N))$ that is an eigenvector under T_p and $\langle n\rangle$. By convention, eigenforms are normalized, meaning the the form is scaled so the first nonzero coefficient of the q-expansion is 1.

The existence of simultaneous eigenforms is nontrivial, but we omit the proof. Another non-obvious property is that eigenforms form an eigenbasis for $S_k(\Gamma_1(N))^{new}$. To determine the eigenvalues of eigenforms, we return to the definitions of T_p and $\langle n\rangle$. Some work demonstrates that for eigenform $f \in S_k(\Gamma_1(N))^{new}$, $T_n f = a_n(f)f$ where $a_n(f)$ is the nth coefficient of the q-expansion of f. With this in mind, the number field that coefficients a_n lie in is referred to as the Hecke eigenvalue field. Further, we state without proof that the coefficients are elements of the ring of integers of said number field. To discuss eigenvalues of $\langle n\rangle$, we need another definition

Definition 2.11. A Dirichlet character modulo N is a homomorphism from from $(\mathbb{Z}/N\mathbb{Z})^*$ to $\mathbb{C}^*$

Dirichlet characters form a group under composition. In fact, this group is precisely the dual group of $(\mathbb{Z}/N\mathbb{Z})^*$, and is denoted $\widehat{(\mathbb{Z}/N\mathbb{Z})^*}$. Dirichlet characters are associated to newforms through the eigenvalues of the $\langle n\rangle$ operator. We define the set of newforms of weight k and level N with character χ as $\{f \in S_k(\Gamma_1(N)) \,|\langle d\rangle f = \chi(d)f \ \forall d \in (\mathbb{Z}/N\mathbb{Z})^*\}$, and denote it $S_k(N,\chi)$.

3 Preliminary notions on representations

3.1 Representations

Definition 3.1. Let G be a group and V be a finite dimensional vector space over a field L. A representation of G on V is an action on V that is linear, namely $\forall\ g \in G$, $x, y \in V$ and $a \in L$, $g(x+y) = gx+gy$ and $g(ax)=a(gx)$. [1]

A representation is equivalent to fixing a group homomorphsim $\rho : G \to GL(V)$ where $GL(V)$ corresponds to $GL_n(L)$ where n =dim(V), the matrix group of invertible $n \times n$ matrices with coefficients in L. [1]

Proposition 3.1. *Let ρ_1 be a representation of group G on a vector space V and let ρ_2 be one on a vector space W. Then there exists a representation on $V \oplus W$.*

Proof. Let ρ_1: $G \to GL_n(L)$ and ρ_2: $G \to GL_m(L)$ where $n = \dim(V)$ and $m = \dim(W)$. Then $n+m = \dim(V \oplus W)$. Let ρ: $G \to GL_{n+m}(L)$ such that $\forall\ g \in G$, $\rho(g) = \begin{pmatrix} \rho_1(g) & 0 \\ 0 & \rho_2(g) \end{pmatrix}$. This is a homomorphism. Let $g_1, g_2 \in G$. $\rho(g_1 g_2) = \begin{pmatrix} \rho_1(g_1 g_2) & 0 \\ 0 & \rho_2(g_1 g_2) \end{pmatrix} = \begin{pmatrix} \rho_1(g_1)\rho_1(g_2) & 0 \\ 0 & \rho_2(g_1)\rho_2(g_2) \end{pmatrix} = \begin{pmatrix} \rho_1(g_1) & 0 \\ 0 & \rho_2(g_1) \end{pmatrix} \begin{pmatrix} \rho_1(g_2) & 0 \\ 0 & \rho_2(g_2) \end{pmatrix} = \rho(g_1)\rho(g_2)$ [1] □

Definition 3.2. Let G be a group. A G-stable subspace is a subspace W such that $\forall\ w \in W$ and $g \in G$, $gw \in W$ under a representation of G on V. [1]

Definition 3.3. A representation of G on V is said to be irreducible if the only G-stable subspaces are 0 and V. [1]

3.2 Prime ideals in a ring of integers

This section outlines factorization of ideals into prime ideals in general ring of integers. Let F be a number field and $\mathcal{O}_F$ be its ring of integers. Then, for any prime p, the principal ideal $p\mathcal{O}_F$ factors as follows:

$$p\mathcal{O}_F = \prod_{i=1}^{r} \mathfrak{p}_i^{e_i}$$

where $\mathfrak{p}_i$ are prime ideals of $\mathcal{O}_F$. [5] If $F/\mathbb{Q}$ is Galois, then all e_i are equal, and its value is called the ramification degree. We say that each $\mathfrak{p}_i$ divides p. r is called the decomposition index and is the number of distinct prime ideal that divides p. The prime p ramifies in F if one of the e_i is strictly bigger than 1. We know that only finitely many primes ramify in a field. [5]

Definition 3.4. The decomposition group of $\mathfrak{p}$ for $\mathfrak{p}$ a prime ideal is $D_\mathfrak{p} = \{\sigma \in Gal(F/\mathbb{Q}) \parallel \sigma(\mathfrak{p}) = \mathfrak{p}\}$. [5]

Remark 3.1. *Let $K = \mathcal{O}_F/\mathfrak{p}_i$. $D_{\mathfrak{p}_i}$ for $\mathfrak{p}_i$ and p as above surjects into $Gal(K/\mathbb{F}_p)$ which is a cyclic group generated by the Frobenius homomorphism corresponding to x goes to x^p.*

If p is unramified in F then, $D_{\mathfrak{p}_i}$ injects into $Gal(K/\mathbb{F}_p)$ for K as above. Then, by the remark, the two are isomorphic. For any $\mathfrak{p}$ dividing p, $\exists$ $\text{Frob}_{\mathfrak{p}}$ $\in D_p$ such that under the isomorphism it corresponds to the Frobenius homomorphism in $Gal(K/\mathbb{F}_p)$ where $K = \mathcal{O}_F/\mathfrak{p}$. This element depends only on $\mathfrak{p}$. However, its conjugacy class, the set $\{gFrob_{\mathfrak{p}}g^{-1}\ \forall g \in Gal(F/\mathbb{Q})\}$, depends only on p. Indeed, if $\mathfrak{p}$, $\mathfrak{q}$ both divide p, then $\exists\ \eta \in Gal(F/\mathbb{Q})$ such that $\eta(\mathfrak{p})$ $= \mathfrak{q}$. Then, $\forall\ \sigma \in D_{\mathfrak{p}}$, $\eta\sigma\eta^{-1}(\mathfrak{q}) = \eta\sigma(\mathfrak{p}) = \eta(\mathfrak{p}) = \mathfrak{q}$. Thus $\eta D_{\mathfrak{p}}\eta^{-1}$=$D_{\mathfrak{q}}$ which implies $\eta Frob_{\mathfrak{p}}\eta^{-1} = Frob_{\mathfrak{q}}$. We denote this conjugacy classes $Frob_p$. [1]

3.3 Galois representations

Let L be a topological field. We will mostly use a finite field with the discrete topology. [1]

Definition 3.5. A n-dimensional Galois representation is a homomorphism ρ: $Gal(\overline{\mathbb{Q}}/\mathbb{Q}) \to GL_n(L)$ for L as above. [5]

Definition 3.6. A n-dimensional continuous Galois representation is a n-dimensional Galois representation that can factor through $Gal(F/\mathbb{Q})$ such that ρ: $Gal(\overline{\mathbb{Q}}/\mathbb{Q}) \to Gal(F/\mathbb{Q}) \to GL_n(L)$ where $F/\mathbb{Q}$ is a finite dimensional Galois extension. [1]

We say a Galois representation ρ is unramified outside S for S a finite set of primes if it factors through $Gal(\overline{\mathbb{Q}}/\mathbb{Q})$. If L is a finite field with the discrete topology, then this simply means that any prime ramifying in F is in S. [1]

3.4 Serre's conjecture

Due to the work of Deligne and Serre, we can associate a Galois representation to any normalized cuspidal eigenform. Let $f \in \text{S}_k(\Gamma_0(N), \chi)$. Let $\mathcal{O}_f$ be the ring of integers of Hecke eigenvalue field K_f, and let $\lambda \in \mathcal{O}_f$ be prime. The Galois representation associated to f is: $\rho_{f,\lambda}$: $Gal(\overline{\mathbb{Q}}/\mathbb{Q}) \to GL_2(\mathcal{O}_f/\lambda)$. $\rho_{f,\lambda}$ is unramified except for primes p such that $p \nmid Nl$ (for l the integer such that $l \mid \lambda$). Considering the conjugacy classes $Frob_p \subset Gal(F/\mathbb{Q})$, the characteristic polynomial of $\rho_{f,\lambda}(Frob_p) = x^2$ - $a_p(f)x$ + $\chi(p)p^{k-1}$ (mod λ) where $a_p(f)$ corresponds to the pth coefficient in the q-expansion of f. [1]

Serre conjectured a partial inverse [1]. His weak modularity conjecture stated that any continuous, odd, 2-dimensional Galois representation, ρ, with image in $GL_2(F)$ for a finite field F arises from a modular form. He further surmised the weight, level, and character this modular form would admit. This

is the strong modularity conjecture (and the reason for the distinction). However, soon after he found a counter-example to the character specification. The strong conjecture (save the portion on character) was proven in 2009 by Khare and Wintenberger. [1] This ρ is proven to be isomorphic to some $\overline{\rho_{f,\lambda}}$. Where $\rho_{f,\lambda}$ is associated to a modular form as described above and $\overline{\rho_{f,\lambda}}$ is $\rho_{f,\lambda}$ reduced modulo a prime. [1]

4 Program

4.1 Weight one forms

Felix Klein classified the finite subgroups of $\mathrm{GL}_2(\mathbb{C})$. [12] Using a similar approach to Klein's work, we are now able to compute the possible images of a finite subgroup $\mathrm{G} \subseteq \mathrm{GL}_2(\overline{\mathbb{F}}_p)$ in the quotient of $\mathrm{GL}_2(\overline{\mathbb{F}}_p)$ with its group of scalar matrices denoted $\mathrm{PGL}_2(\overline{\mathbb{F}}_p)$. Also, G must act irreducibly on $\overline{\mathbb{F}}_p^2$. The possible images are either isomorphic to a finite dihedral group, A_4, S_4, or A_5 or conjugate to one of $\mathrm{PGL}_2(k)$ or $\mathrm{PSL}_2(k)$ for a finite subfield k of $\overline{\mathbb{F}_p}$. [1]

Let G be the image of a Galois representation $\rho : \mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q}) \to \mathrm{GL}_2(\overline{\mathbb{F}}_p)$. Let ϕ be the surjection from $\mathrm{GL}_2(\overline{\mathbb{F}}_p)$ to $\mathrm{PGL}_2(\overline{\mathbb{F}}_p)$. The kernel of $\phi \circ \rho$ is a finite Galois extension $\mathrm{F}/\mathbb{Q}$. If the image of G under ϕ is dihedral, then the kernel is a quadratic extension over $\mathbb{Q}$ corresponding to the index 2 subgroup in the dihedral group. [1]

A result from Henri Carayol states that the newforms that are counterexamples to Serre's conjecture must be in $\mathrm{S}_k(\Gamma_1(N))$ and have a mod 3 representations such that their image is dihedral like explained above. Also, the kernel must correspond to $\mathbb{Q}(\sqrt{-3})$. However, it is computationally heavy to look at all possible newforms in $\mathrm{S}_k(\Gamma_1(N))$ when k$\geq$ 2. Nonetheless, those newforms will be congruent to a weight one form of level $\Gamma_1(3N)$. Therefore, we need only check the congruence of the newforms in $\mathrm{S}_1(\Gamma_1(3N))$, a smaller vector space. [1]

Since weight one forms aren't implemented in Sage, we downloaded the defining polynomials, and labels of all weight one forms from the L-Functions and Modular Forms Database (LMFDB). The main function iterates through the list of labels and automatically downloads the q-expansion of suspected counterexamples to Sage, and uses the defining polynomial to construct the Hecke eigenvalue field. We considered forms with level $3N$ for N prime and Dirichlet

character of order divisible by 3. From this, we constructed the corresponding space of newforms of higher weight and level N and tested congruence.

4.2 Carayol Conditions

Henri Carayol proved in his paper 'Sur les Représentations galoisiennes modulo l attachées aux formes modulaires' that any such counterexample must satisfy a set of conditions. The Carayol conditions are satisfied (with regard to primes above 3) when the image of the Galois representation ρ is dihedral and the quadratic extension is $\mathbb{Q}(\sqrt{-3})$. Modular form g satisfies the Carayol condition if for every coefficient of its q-expansion a_i with $i \equiv 2 \mod 3$ and i not dividing the level, a_i is congruent to 0 modulo some prime P such that $P \cap \mathbb{Z} = 3\mathbb{Z}$. Furthermore, every a_i with $i \equiv 1 \mod 3$ and i not dividing the level is not congruent to 2 modulo p. We only need to check when i is prime, since we can construct the other coefficients from the prime-indexed ones. The first condition assures that ρ is either reducible or has dihedral projective image and the second one assures it is not reducible.

To check this condition, the program first factors the principal ideal generated by 3 into prime ideals I in $\mathcal{O}_g$, the ring of integers of the coefficients. For each ideal, it constructs the residue field $R := \mathcal{O}_g/I$ and creates the set of all homomorphisms between R and the standard finite field of the same cardinality. It, then, iterates through this set. For each a_i with $i \equiv 2 \mod 3$, the program takes its image under the homomorphism. If it is nonzero, the program advances to the next homomorphism or ideal, and returns false if all have been tested. If all such a_i are mapped to 0, it returns true.

4.3 Sturm bound

The Sturm bound is an upper bound relating to the index of coefficients of the q-expansion of modular forms. The q-expansion of two distinct newforms will necessarily differ at some index less than the Sturm bound, therefore to test congruence, one only needs to check prime indexed coefficients with the index less than the bound. It is calculated for all function in the space $\mathrm{M}_k(\mathrm{N}, \chi)$ by the formula $\lfloor \frac{km}{12} \rfloor$ where k is the weight and $m = [SL_2(\mathbb{Z}):\Gamma_0(\mathrm{N})]$. It can also be calculated using the equivalent formula $m = \mathrm{N}\prod_{p|N}(1 + \frac{1}{p})$. [10]

4.4 Checking congruence

To determine if the Galois representation generated by a weight one form would be isomorphic to the Galois representation generated by the newform conjectured by Serre, the algorithm checks if there is a newform congruent to the weight one form modulo some prime above 3. The space of newform created depended on the specified weight. If the weight is even, the Dirichlet character is the primitive character of the one of weight one form composed with the non trivial homomorphism from $(\mathbb{Z}/3\mathbb{Z})^*$ to $\mathbb{C}$, and the level to check the level of the weight one form divided by 3. If the weight was odd then the Dirichlet character and the level is the same as the one of the weight one form. Looping through the space just created, it checks for the newform with Dirichlet character order not divisible by 3, if there is a congruence. If so, the weight one form is not a counterexample. Otherwise, if no such newform existed, the weight one form is a counterexample.

Given a newform orbit f, the q-expansion of a weight one form g, and the ring of integers $\mathcal{O}_g$ the coefficients of g lie in, the function checkCongruence returns true if some form in f is congruent to g for some prime above 3. First the Hecke eigenvalue field of f and then its corresponding ring of integers $\mathcal{O}_f$ is computed. Since there is no known polynomial-time algorithm to compute the ring of integers of a number field, the function is timed, and automatically continues to the next weight one form in the list after 20 minutes. Due to the implementation of rings of integers in Sage, cases where $\mathcal{O}_f = \mathbb{Z}$ or $\mathcal{O}_g = \mathbb{Z}$ are handled separately. [14] [15] The most general case, where $\mathcal{O}_f, \mathcal{O}_g \neq \mathbb{Z}$, proceeds as follows. Principal ideal $3\mathbb{Z}$ is factored in $\mathcal{O}_f$ into prime ideals I_f which are then iterated through. For each ideal, the residue field $\mathcal{O}_f/I_f$ and a homomorphism ϕ from it to the standard finite field of the same cardinality are created. Next the prime ideals dividing $3\mathbb{Z}$ in $\mathcal{O}_g$ are computed, and again iterated through. Each residue field $\mathcal{O}_g/I_g$ is constructed, as well as the set of all homomorphisms S from $\mathcal{O}_g$ to the standard finite field of cardinality $|\mathcal{O}_g/I_g|$. Since all standard finite fields of the same character have natural inclusions in sage, congruence of coefficients a_f and a_g can be tested by comparing their images under ϕ and some $\eta \in S$ respectively. If there exists some combination of ideals I_f and I_g and some homomorphism η such that all prime coefficients up to the Sturm bound have the the same image, the forms are congruent, and the function returns true.

An alternate version of the function counts the number of homomorphisms η that yield a congruence, thereby counting the number of newforms in the

orbit congruent to g, rather than checking the newform orbit as a whole. Another modification consists of finding every modular form in $S_k(\Gamma_1(\mathrm{N}), \chi)$ for k even or in $S_k(\chi, \Gamma_1(3\mathrm{N}))$ for k odd such that they are congruent to the weight one newform g and output the order of their Dirichlet character. The modified code proceeds as follows: Let the order of Dirichlet character of g be r. We require that $3 \nmid r$ and that $r \mid$ (N-1). We know the possible order of the Dirichlet character for f congruent to g are r, $3r$, 3^2r, ..., 3^kr where $N - 1 = 3^kM$ and $3 \nmid M$. [1]

5 Pseudocode

See Appendix A for the code in SageMath [14] [15]

Algorithm 1 Main function

```
Result: list of the LMFDB labels of the examples
Input: start_index, end_index, weight
current ← start_index
while current is not equal to end_index do
    label ← mf_newform.labels[current]
    level ← level_of_label
    if 3 | level and level/3 is prime then
        Create make_data.py
        Download from lmfdb.org download.text
        Write in make_data.py download.text
        Import make_data.py
        g ← make_data.make_data.()
        g_ringInt ← ring_of_integer_of_g
        sturm ← calculate_sturm_bound
        if 3 | order of Dirichlet character of g then
            if g respects Carayol condition then
                if weight is even then
                    Let alpha be the non trivial homomorphism from (ℤ/3ℤ)*
                     to ℂ
                    psi ← Dirichlet_character_g * alpha
                    chi ← primitive_character_of_psi
                    forms ← S_k(chi, Γ(N))
                else
                    psi ← Dirichlet_character_g
                    forms ← S_k(psi, Γ(3N))
                end
                for f in forms do
                    if 3 ∤ order of Dirichlet character of f then
                        if f is congruent to g then
                            good_g ← False
                        end
                    end
                end
                if good_g then
                    output label of g
                end
            end
        end
    end
    Increment current
end
```

Algorithm 2 Check congruence function

Result: boolean saying if the two from are congruent

Input: f, g, g_ringInts, sturm_bound

g is the weight 1 modular form and f is the modular form we want to check if it is congruent to g

$f_ringInts \leftarrow ring_of_integer_of_f$

$ideals_f \leftarrow ideals_of_f_ringInts_above_3$

$ideals_g \leftarrow ideals_of_g_ringInts_above_3$

for *ideal_f in ideals_f* **do**

  $residue_field_f \leftarrow ideal_f.residue_field$

  Φ is a homomorphism from residue_field_f to the Standard finte field of the same cardinality

  for *ideal_g in ideals_g* **do**

    $residue_field_g \leftarrow ideal_g.residue_field$

    HomSet is the set of homomorphism from residue_field_g to the Standard finte field of the same cardinality

    for Ψ *in HomSet* **do**

    end

    good $\leftarrow True$

    for *p in list of primes from 1 to sturm_bound* **do**

      if Φ*(f_ringInts(f[p]))* - Ψ*(g_ringInts(g[p]))* $\neq$ *0* **then**

        good $\leftarrow False$

      end

      if *good* **then**

        return *True*

      end

    end

  end

end

return *False*

Algorithm 3 Check Carayol

Result: True if g satisfies the Carayol conditions, False otherwise

Input: g, $gInts, P, level, sturm_bound$

```
ideals ← prime ideals above P
for ideal in ideals do
    good ← True
    residue_field ← ideal_g.residue_field
    Φ ← a homomorphism from residue_field_g to the standard finite field of
    the same cardinality
    i ← 0
    primes ← Primes from 1 to Sturm bound
    while i less than length of primes and good is true do
        if p_i% 3 = 2 and p_i does not divide level then
            if Φ(g_ringInts(p_i)) ≠ 0 then
                good ← False
            end
        end
        if p_i% 3 = 1 and p_i does not divide level then
            if Φ(g_ringInts(p_i))  0 then
                good ← False
            end
        end
        i += 1
    end
    if good then
        return True
    end
end
return False
```

6 Examples

To list the examples, we are using LMFDB labels. They are of the following format N.K.a.x where N corresponds to the level, k to the weight, N.a to the Galois orbit of the Dirichlet character and x to the Galois orbit of the newform. [9] In the next table, 'Weight' is the weight of the newform f. 'Maximum level' is the level at which we interrupted the program. 'Counterexamples' is the LMFDB label of the counterexamples. Finally, 'Skipped labels' is the LMFDB labels of the newforms skipped by the 20 minutes timer.

Table 1: Counterexamples to the character in Serre's Conjecture

Weight	Maximum level	Counterexamples		Skipped Newform Orbits	
2	939	39.1.d.a,	93.1.l.a,	309.1.l.a,	417.1.l.a,
		111.1.d.a,	129.1.l.a,	453.1.t.a,	471.1.n.a,
		183.1.d.a,	183.1.l.a,	471.1.o.a,	579.1.o.a,
		183.1.n.a,	201.1.k.a,	579.1.s.a,	597.1.p.a,
		219.1.c.a,	219.1.f.a,	633.1.p.a,	633.1.z.a,
		237.1.k.a,	291.1.c.a,	669.1.l.a,	687.1.n.a,
		291.1.g.a,	291.1.k.a,	687.1.p.a,	723.1.m.a,
		291.1.o.a,	381.1.m.a,	723.1.n.a,	723.1.o.a,
		453.1.j.a,	471.1.d.a,	723.1.w.a,	723.1.bc.a,
		471.1.d.a,	543.1.m.a,	813.1.m.a,	831.1.n.a,
		543.1.o.a,	579.1.c.a,	831.1.p.a,	849.1.l.a,
		579.1.g.a,	633.1.m.a,	921.1.p.a,	939.1.s.a,
		723.1.b.a,	723.1.g.a,	939.1.t.a,	939.1.v.a,
		831.1.d.a,	939.1.c.a,	993.1.l.a,	993.1.p.a,
		939.1.g.a		993.1.ba.a	
3	300	None		93.1.l.a,	129.1.l.a,
				183.1.l.a,	183.1.n.a,
				201.1.k.a,	219.1.f.a,
				237.1.k.a,	291.1.g.a,
				291.1.k.a,	291.1.c.a,
				291.1.g.a,	291.1.k.a,
				291.1.o.a	
4	312	None		183.1.l.a,	237.1.k.a,
				291.1.g.a,	291.1.k.a,
				291.1.o.a	
6	543	None		129.1.l.a,	183.1.l.a,
				183.1.n.a,	201.1.k.a,
				219.1.f.a,	237.1.k.a,
				291.1.c.a,	291.1.k.a,
				309.1.l.a,	327.1.d.b,
				327.1.d.d,	381.1.m.a,
				417.1.l.a,	453.1.j.a,
				453.1.t.a,	471.1.d.a,
				471.1.d.b,	471.1.n.a,
				471.1.n.a,	471.1.o.a,
				543.1.d.b	

In the next table, 'Counterexamples' and 'Skipped labels' are similar to the previous table. 'Order' is the Dirichlet character order of the newforms congruent to the counterexample.

Table 2: Counterexamples found when testing with weight 2 newforms and the Dirichlet character order of these newform

Counterexample	Order	Skipped Newform Orbits
39.1.d.a	6	201.1.k.a
93.1.l.a	15	219.1.f.a
111.1.d.a	18	237.1.k.a
129.1.l.a	21	291.1.o.a
183.1.d.a	6	309.1.l.a
183.1.l.a	30	381.1.m.a
183.1.n.a	15	
219.1.c.a	18	219.1.c.a for the order 36
291.1.c.a	6	291.1.c.a for the order 48
291.1.g.a	12	291.1.g.a for the order 48
291.1.k.a	24	291.1.k.a for the order 48
327.1.d.b	2, 6, 18	327.1.d.b for the order 54
327.1.d.d	2, 6, 18	327.1.d.d for the order 54

Table 3 records the number of newforms in each orbit that are congruent to the corresponding weight one form. This is computed by counting the number of homomorphisms from $\mathcal{O}_g$ to a residue field that results in a congruence. N/A refers to forms that were skipped.

Table 3: Number of congruences

Weight	Newform Orbit	Congruences Found
2	39.1.d.a	1
2	93.1.l.a	1
2	111.1.d.a	1
2	129.1.l.a	1
2	183.1.d.a	1
2	183.1.l.a	1
2	183.1.n.a	1
2	201.1.k.a	N/A
2	219.1.c.a	1
2	219.1.f.a	1
2	237.1.k.a	1

Finally, we tested whether or not 39.1.d.a and 93.1.l.a were still counterexamples for higher weights. After testing each form against newform orbits of even weights from weight 4 to 20 and 4 to 12 respectively, neither form remained a counterexample.

Appendices

A Original code

```
from wrapt_timeout_decorator import *
from sage.modular.Dirichlet import DirichletGroup
from sage.modular.Dirichlet import DirichletCharacter
import requests
import importlib
import mf_newforms as mf
import math
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
url_part = 'https://www.lmfdb.org/ModularForm/
                GL2/Q/holomorphic/download_qexp/'

def main_function(start_index, end_index, k):
    code_file = open('make_data.py', 'w')
    code_file.close()
    import make_data
    list_of_examples = []
    list_of_index=[]
    current_index = start_index
    url = url_part + mf.labels[current_index]

    while current_index <= end_index:
        print('current label, index: ' + mf.labels[current_index]
            + ', ' + str(current_index))
        if mf.data[current_index][0]%3 != 0:
            current_index += 1
            continue
        if not (Integer(mf.data[current_index][0]/3)).is_prime():
            current_index += 1
            continue

        url = url_part + mf.labels[current_index]
        download = requests.get(url)
        code_file = open('make_data.py', 'w')
        code_file.write('char_values = {} \n')
        code_file.write('var = ""')
```

```
code_file.write(download.text)
code_file.close()
prepend_text('make_data.py', '\n    global char_values \n
    global var', 'def make_data():')
prepend_text('make_data.py', '\n
    var=vars(K)["_NumberField_generic__gen"]','an[1] = 1')
importlib.reload(make_data)

gqexp = make_data.make_data()
poly=make_data.poly_data
gInts = getGInts(current_index, poly)
level = mf.data[current_index][0]
sturm = sturm_bound(level, 1)
char_values = make_data.char_values
gen = make_data.hecke_ring_character_values
var=make_data.var
good=True

if gcd(order_Dirichlet(char_values, gen),3)!=1:
    good=False
else:
    if not checkG(gqexp, gInts, 3, level, sturm):
        good=False
    else:
        if k%2 == 0:
            fLevel = Integer(level/3)
            parent_psi = DirichletGroup(level)
            psi=DirichletCharacter(parent_psi,
                                   get_chi(gen, var))
            parent_chi=DirichletGroup(fLevel)
            chi=psi.primitive_character()
            try:
                forms=getNewforms(
                    chi,
                    k, current_index, poly)
            except:
                good=False
                code_file = open('output.txt', 'a')
                code_file.write('SKIPPED FOR NEWFORM.
                    label, index: ' +
                    str(mf.labels[current_index])
                    + ', ' + str(current_index) + '\n')
```

```
                code_file.close()
                current_index += 1
                continue
        else:
            fLevel = level
            parent = DirichletGroup(fLevel)
            chi=DirichletCharacter(parent,
                Dirichlet(gen, var))
            print('chi done,
                generating space of newforms')
            try:
                forms = getNewforms(chi,
                    k, current_index, poly)
            except:
                good=False
                code_file = open('output.txt', 'a')
                code_file.write('SKIPPED FOR NEWFORM.
                    label, index: ' +
                    str(mf.labels[current_index])
                    + ', ' + str(current_index) + '\n')
                code_file.close()
                current_index += 1
                continue

        for form in forms:
            if form.character().order() % 3 !=0:
                s_f = sturm_bound(fLevel, k)
                s_g = sturm_bound(level, 1)
                bound = min(s_f, s_g)
                try:
                    if checkCongruence(form, gqexp, gInts,
                        bound, mf.labels[current_index]):
                        good=False
                        break
                except:
                    good=False
                    code_file = open('output.txt', 'a')
                    code_file.write('SKIPPED FOR CONG.
                        label, index: ' +
                        str(mf.labels[current_index])
                        +', '+ str(current_index) +'\n')
                    code_file.close()
```

```
                                current_index += 1
                                continue

        if good:
            print('SUCCESS: ' + mf.labels[current_index])
            outputter = open('output.txt', 'a')
            outputter.write('SUCCESS label, index: '
                + mf.labels[current_index] + ', '
                + str(current_index) + '\n')
            outputter.close()
            list_of_examples.append(mf.labels[current_index])
            list_of_index.append(current_index)

        current_index += 1
    return list_of_examples

@timeout(1200)
def getNewforms(chi, k, current_index, poly):
    print('generating newforms')
    N = Newforms(chi, k, getGField(current_index, poly), 'a')
    print('newforms generated. dimemsion ' + str(len(N)))
    return(N)

def getGInts(current_index, poly_data):
    ci = poly_data
    if type(ci) == list:
        poly = x - x
        for i in range(len(ci)):
            poly = poly + ci[i] * x ^ i
        Field = NumberField(poly, 'a')
    else:
        Field = CyclotomicField(ci)
    return(Field.ring_of_integers())

def getGField(current_index, poly_data):
    ci = poly_data
    if type(ci) == list:
        poly = x - x
        for i in range(len(ci)):
            poly = poly + ci[i] * x ^ i
        Field = NumberField(poly, 'a')
    else:
```

```
            Field = CyclotomicField(ci)
165     return Field

    def prepend_text(file, text, after=None):
        f_read = open(file, 'r')
        buff = f_read.read()
170     f_read.close()
        f_write = open(file, 'w')
        inject_pos = 0
        if after:
            pattern = after
175         inject_pos = buff.find(pattern)+len(pattern)
        f_write.write(buff[:inject_pos] + text + buff[inject_pos:])
        f_write.close()

    def Dirichlet(hecke, var):
180     values=[]
        for gen in hecke:
            if type(gen[1][0])==list:
                value=gen[1][0][0]* var ** gen[1][0][1]
                values.append(value)
185         else:
                value=gen[1][0]
                values.append(value)
        return values

190 def get_chi(hecke, var):
        values=[]
        for gen in hecke:
            if type(gen[1][0])==list:
                value=gen[1][0][0]* var ** gen[1][0][1]
195         else:
                value=gen[1][0]
            if Integer(gen[0]) % 3==2:
                value=value * -1
            values.append(value)
200     return values

    def order_Dirichlet(char_values, gen):
        order = 1
        increase = False
205     value=[1]*len(gen)
```

```
    for i in range(len(gen)):
        factor=Integer(gen[i][0]).factor()
        for f in list(factor):
            value[i]=value[i]*char_values[f[0]]**f[1]
    while 1 == 1:
        for i in range(len(gen)):
            if value[i] ** order != 1:
                increase = True
        if increase:
            order += 1
            increase = False
        else:
            break
    return order

def sturm_bound(level, weight):  # calculate sturm bound
    primes = list(factor(level))
    product = 1
    for prime in primes:
        m = 1 + 1 / (prime[0])
        product = product * m
    product = product * level
    sturm = floor(level * weight / 12)
    return sturm

@timeout(1200)
def checkCongruence(f, g, g_ring_of_ints, bound):
    F = f.hecke_eigenvalue_field()
    O_f = F.ring_of_integers()

    if O_f != IntegerRing():
        f_ideals = list(O_f.ideal(3).factor())
        for f_ideal in f_ideals:
            f_residue = f_ideal[0].residue_field()
            Phi = f_residue.Hom(GF(f_residue.cardinality()))[0]
            g_ideals = g_ring_of_ints.ideal(3).factor()
            for i in range(len(g_ideals)):
                g_residue = g_ideals[i][0].residue_field()
                homSet = g_residue.Hom(
                        GF(g_residue.cardinality()))
                for j in range(len(list(homSet))):
                    Psi = homSet[j]
```

```
                    good = True
                    for i in prime_range(1,bound):
                        if Phi(O_f(f[i])) -
                                Psi(g_ring_of_ints(g[i])) != 0:
                            good = False
                    if good:
                        return True
        return False

    else:
        f_ideals = O_f.ideal(3)
        Field = NumberField(x, 'a')
        if g_ring_of_ints != Field.ring_of_integers():
            g_ideals = g_ring_of_ints.ideal(3).factor()
            for g_ideal in list(g_ideals):
                g_residue = g_ideal[0].residue_field()
                hom = g_ring_of_ints.hom(g_residue)
                homSet = g_residue.Hom(
                        GF(g_residue.cardinality()))
                for j in range(len(list(homSet))):
                    Psi = homSet[j]
                    good = True
                    for i in prime_range(1, bound):
                        a = (g_ring_of_ints(g[i]))
                        if Integer(f[i]) % 3 - Psi(hom((a))) != 0:
                            good = False
                    if good:
                        return True
            return False
        else:
            good = True
            for i in prime_range(1, bound):
                if Integer(f[i]) % 3 - Integer(g[i]) % 3 != 0:
                    good = False
            return good

def checkG(g, gInts, P, level, sturm_bound):
    ideals = list(gInts.ideal(P).factor())
    for ideal in ideals:
        good = True
        F = ideal[0].residue_field()
        Phi = F.Hom(GF(F.cardinality()))[0] #canonical
```

```
290         i = 0
            while i < len(prime_range(1, sturm_bound)) and good:
                if  (not prime_range[i].divides(level)) and
                        (prime_range[i] % 3 == 2):
                    if Phi(gInts(g[i])) != 0:
295                     good = False
                i += 1
            if good:
                return True
        return False
```

B Instructions on downloading weight one form data

1. Go to this website : https://www.lmfdb.org, access the main page of the L-functions and modular forms database
2. In the index on the left of the page, under 'Modular forms', click 'Classical'
3. Then select '1' in the 'Browse newforms By weight' section
4. Beside the 'Result', click 'Download to' and select 'SageMath'

References

[1] Allen, P. (May 15, 2023 - August 3, 2023), Personal communications.

[2] Borcherds, R. E. (March 20, 2021). Modular forms: Introduction. [Video]. YouTube. https://www.youtube.com/watch?v=f8pjt9ivjjc&list=PL8yHsr3EFj51HisRtNyzHX-Xyg6I3Wl2F.

[3] Boston, N. (2003) The proof of Fermat's last theorem. University of Wisconsin - Madison. https://people.math.wisc.edu/ nboston/869.pdf.

[4] Carayol, H. (1989, December). Sur les Représentations galoisiennes modulo l attachées aux formes modulaires. Duke Mathematical Journal.

[5] Diamond, F. & Shurman, J. (2000). A First Course in Modular Form. Springer.

[6] Edixhoven, B. (September 23, 2005). Introduction to Serre's conjecture. https://math.mit.edu/ drew/edix1.pdf.

[7] The Editors of Encyclopaedia Britannica. (n.d.). Fermat's last theorem. Encyclopaedia Britannica. https://www.britannica.com/science/Fermats-last-theorem.

[8] International Centre for Theoretical Sciences. (December 29, 2020). Serre's Conjecture for GL_2 over Totally Real Fields (Lecture 1) by Fred Diamond. [Video]. YouTube. https://www.youtube.com/watch?v=OFJ11TJY41g.

[9] LMFDB. (n.d.). The L-functions and modular forms database. https://www.lmfdb.org.

[10] LMFDB. (n.d.). Sturm bound (reviewed). https://www.lmfdb.org/knowledge/show/cmf.sturm_bound.

[11] Milne, J. S. (1990) Modular Functions and Modular Forms. Math 678. University of Michigan. https://www.jmilne.org/math/CourseNotes/MF110.pdf.

[12] Prasad, D. Yogananda, C. S. (February 23, 2007). A Report on Artin's holomorphy conjecture. http://www.math.tifr.res.in/ dprasad/artin.pdf.

[13] Ribet, K. A. and Stein, W. (2010, November 23). Lectures on Serre's conjectures. American Mathematical Society. https://wstein.org/papers/serre/ribet-stein.pdf.

[14] SageMath. (2005-2023). Sage. Github. https://github.com/sagemath/sage.

[15] SageMath. (2005-2023). Sage Reference Manual. The Sage Development Team. https://doc.sagemath.org/html/en/reference/index.html.

[16] Serre, J.-P. (1973). Course in arithmetic. Springer.

[17] Serre, J.-P. (1987). Lettre à J.-F. Mestre. Contemporary Mathematics : Current Trends in Arithmetical Algebraic Geometry. pp.263–268. American Mathematical Society. https://www.ams.org/books/conm/067/conm067.pdf.

[18] Serre, J.-P. (1987). Sur les représentations modulaires de degré 2 de Gal($\overline{\mathbb{Q}}/\mathbb{Q}$). Duke Mathematical Journal. https://www.college-de-france.fr/media/jean-pierre-serre/UPL5835292064138487263_Serre_Repr.modulaires_Galois.pdf.

[19] Stein, W. (2004, May 06). A Brief Introduction to Classical and Adelic Algebraic Number Theory : Rings of Algebraic Integers. Willian Stein. https://wstein.org/papers/ant/html/node14.html.

[20] Sutherland, A. (2017). Modular forms and L-functions Lecture 25. 18.783 Elliptic Curves. Massachusetts Institute of Technology. https://math.mit.edu/classes/18.783/2017/LectureNotes25.pdf.

[21] Vidéos éducatives. (May 15, 2014)M. odular Forms & Hecke Operators Mathematics Lecture. [Video]. YouTube. https://www.youtube.com/watch?v=svbMHtmhMRg.